LAB MANUAL

Mobile Application Development

NotificationManager

Android Application

A Complete Guide for Understanding Android Notification Channels, Navigation Components, ViewBinding & Runtime Permissions

Platform:	Android (Java)
Min SDK:	API 26 (Android 8.0 Oreo)
Target SDK:	API 36
Build Tool:	Gradle with Kotlin DSL
Patterns:	MVVM / Single Activity

Department of Artificial Intelligence And Data Science

Academic Year 2025–26

1. Introduction

The NotificationManager Android application is a well-structured, fully functional teaching project designed to demonstrate several fundamental concepts in modern Android development. It was created using Android Studio with Java as the programming language.

1.1 Purpose of This Application

This application demonstrates how Android developers can:

- Create and register a Notification Channel (required on API 26+).
- Post system notifications using NotificationCompat.Builder.
- Handle runtime permissions for posting notifications (Android 13 / API 33+).
- Navigate between screens using the Jetpack Navigation Component.
- Bind UI views safely using ViewBinding (no more findViewById).
- Build a responsive, Material Design 3 interface using CoordinatorLayout, AppBar, and FAB.
- Support multiple languages (English, Hindi, Kannada, Telugu) through locale switching.

1.2 Key Technologies Used

Technology	Library / Class	Purpose
Notification System	NotificationManager, NotificationCompat	Create & post notifications
Navigation	NavController, NavHostFragment	Screen-to-screen navigation
ViewBinding	ActivityMainBinding, FragmentFirstBinding	Safe view access
Material Design 3	MaterialToolbar, FAB, Snackbar	UI components
Permission API	ActivityResultLauncher, ContextCompat	Runtime permissions
Localization	Resources, Configuration, Locale	Multi-language support

1.3 Learning Outcomes

After studying and running this application, students will be able to:

1. Understand the full lifecycle of an Android application using AppCompatActivity.
2. Explain the purpose and creation of Notification Channels.
3. Request and handle runtime permissions using the modern ActivityResultLauncher API.
4. Navigate between Fragment screens using Jetpack Navigation.
5. Use ViewBinding to access UI elements without null-pointer risk.
6. Implement runtime locale switching for multi-language apps.
7. Read and interpret Android Manifest, Gradle configuration, and resource files.

2. Project Structure (Structural View)

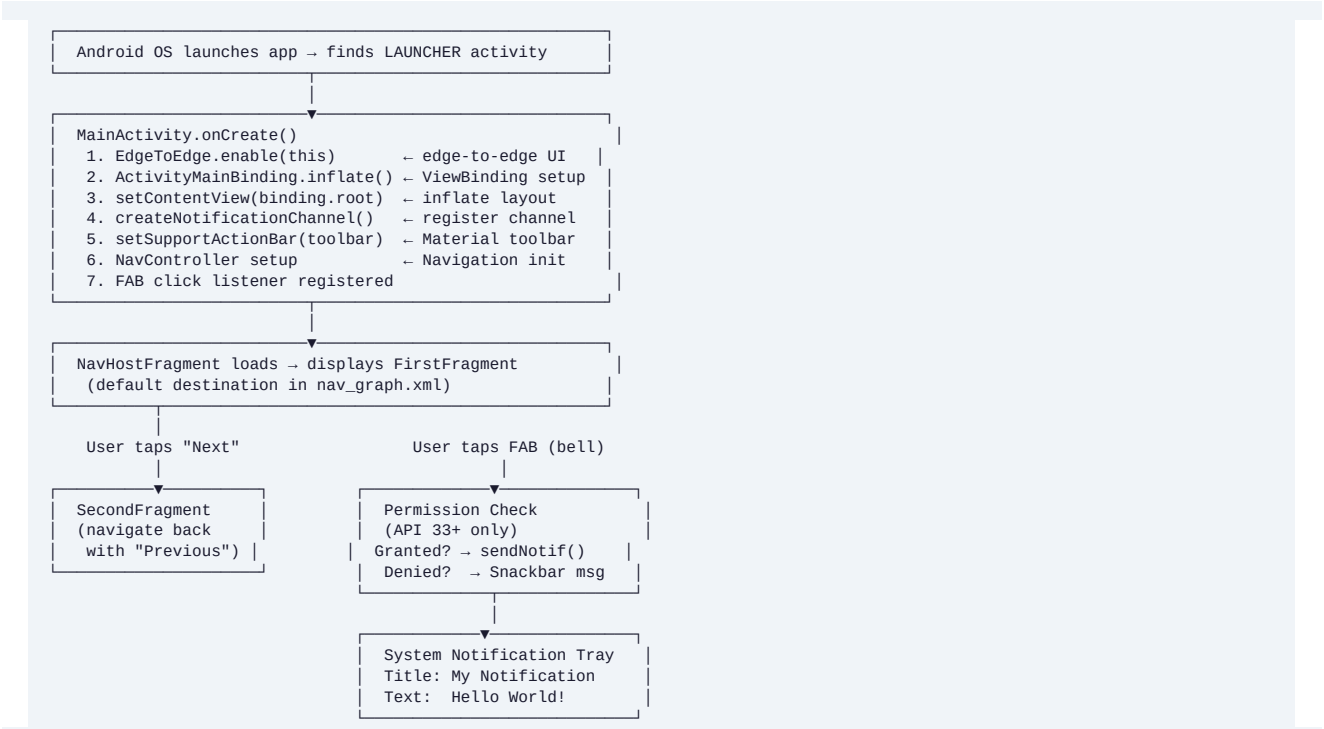
The following tree shows every file relevant to studying this application. Build output folders have been omitted for clarity.

```
NotificationManager/                      ← Root Project
├── build.gradle.kts                       ← Top-level Gradle (plugin declarations)
├── settings.gradle.kts                    ← Module inclusion & repository config
├── gradle/
│   ├── libs.versions.toml                 ← Version Catalog (all dependency versions)
│   └── wrapper/gradle-wrapper.properties  ← Gradle wrapper version (9.4.1)
│
└── app/                                   ← Application Module
    ├── build.gradle.kts                   ← App-level Gradle (plugins, deps, config)
    └── src/main/
        ├── AndroidManifest.xml            ← App manifest (permissions, activities)
        ├── java/com/example/notificationmanager/
        │   ├── MainActivity.java          ← Entry point (notifications + nav setup)
        │   ├── FirstFragment.java         ← Screen 1 (navigation target)
        │   └── SecondFragment.java        ← Screen 2 (navigation target)
        └── res/
            ├── layout/
            │   ├── activity_main.xml      ← Root layout (CoordinatorLayout + FAB)
            │   ├── content_main.xml       ← NavHostFragment container
            │   ├── fragment_first.xml     ← First screen UI
            │   └── fragment_second.xml    ← Second screen UI
            ├── menu/menu_main.xml         ← Options menu (Settings, Language)
            ├── navigation/nav_graph.xml   ← Navigation graph (routes)
            ├── values/
            │   ├── strings.xml            ← English string resources
            │   ├── colors.xml             ← App colour palette
            │   ├── themes.xml             ← App theme (Material3 DayNight)
            │   └── dimens.xml             ← Dimension values (e.g. FAB margin)
            ├── values-hi/strings.xml      ← Hindi translations
            ├── values-kn/strings.xml      ← Kannada translations
            ├── values-te/strings.xml      ← Telugu translations
            └── values-night/themes.xml    ← Dark mode theme overrides
```

> 📌 **Key Concept — Module Structure**
> Android projects are organized as modules. The 'app' module contains all application source code and resources. The root project files configure Gradle's build system. The separation allows multiple modules (e.g., library modules) in larger projects.

3. Application Flow (Top to Bottom)

The diagram below shows the complete execution path from app launch to the user receiving a notification.



3.1 Menu Flow

The options menu (three-dot icon) provides two additional features:

- Settings — Registered menu item (no action implemented, placeholder for future use).
- Change Language — Opens an AlertDialog with 4 language options. On selection, setLocale() updates the app-level Configuration and calls recreate() to hot-reload all string resources.

4. Configuration Files

4.1 settings.gradle.kts — Root Settings

This file is the Gradle settings script. It tells Gradle which modules belong to the project and where to find plugins/dependencies.

```
pluginManagement {
    repositories {
        google {
            content {
                includeGroupByRegex("com\\.android.*")
                includeGroupByRegex("com\\.google.*")
                includeGroupByRegex("androidx.*")
            }
        }
        mavenCentral()
        gradlePluginPortal()
    }
}
plugins {
    id("org.gradle.toolchains.foojay-resolver-convention") version "1.0.0"
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "NotificationManager"
include(":app")
```

> 📘 **Line-by-Line Explanation**
> pluginManagement → tells Gradle where to look for build plugins (e.g., AGP, Kotlin plugin).
> dependencyResolutionManagement → enforces that all dependency repos are declared here, not in modules.
> include(":app") → registers the 'app' subfolder as a subproject/module.

4.2 build.gradle.kts (Root) — Top-Level Build File

```
plugins {
    alias(libs.plugins.android.application) apply false
}
```

This root-level file declares the Android Gradle Plugin (AGP) but does not apply it. The apply false means the plugin is available for sub-modules to use but is not activated at the root level.

4.3 app/build.gradle.kts — Module Build File

This is the most important Gradle file — it configures the application build:

```
plugins {
    alias(libs.plugins.android.application)   // applies AGP
}

android {
    namespace = "com.example.notificationmanager"
    compileSdk { version = release(36) { minorApiLevel = 1 } }

    defaultConfig {
        applicationId = "com.example.notificationmanager"
        minSdk = 26          // Android 8.0 Oreo minimum
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(getDefaultProguardFile(...), "proguard-rules.pro")
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
```

```
    }
    buildFeatures {
        viewBinding = true   // enables ViewBinding code generation
    }
}

dependencies {
    implementation(libs.activity.ktx)
    implementation(libs.appcompat)
    implementation(libs.constraintlayout)
    implementation(libs.material)
    implementation(libs.navigation.fragment)
    implementation(libs.navigation.ui)
    testImplementation(libs.junit)
    androidTestImplementation(libs.espresso.core)
    androidTestImplementation(libs.ext.junit)
}
```

Setting	Meaning
minSdk = 26	App runs only on Android 8.0+. This is required because Notification Channels were introduced in API 26.
viewBinding = true	Gradle generates a Binding class for each layout XML, enabling type-safe view access.
isMinifyEnabled = false	ProGuard/R8 code shrinking is disabled in release builds (suitable for development/demo).
JavaVersion.VERSION_11	Source and target compatibility set to Java 11, enabling lambda expressions and modern syntax.

4.4 gradle/libs.versions.toml — Version Catalog

The Version Catalog centralizes all dependency versions in one place, preventing version conflicts between modules.

```
[versions]
agp                = "9.2.1"       # Android Gradle Plugin
appcompat          = "1.7.1"
material           = "1.14.0"
constraintlayout   = "2.2.1"
activityKtx        = "1.13.0"
navigationFragment = "2.9.8"
navigationUi       = "2.9.8"
junit              = "4.13.2"

[libraries]
appcompat          = { group = "androidx.appcompat", name = "appcompat", version.ref = "appcompat" }
material           = { group = "com.google.android.material", name = "material", version.ref = "material" }
navigation-fragment = { group = "androidx.navigation", name = "navigation-fragment", ... }
navigation-ui       = { group = "androidx.navigation", name = "navigation-ui", ... }

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
```

5. Resource Files (XML — values/)

5.1 AndroidManifest.xml

The Manifest is the control file of the Android application. It declares permissions, components, and metadata.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
          xmlns:tools="http://schemas.android.com/tools">

    <!-- Permission required on Android 13+ (API 33) to post notifications -->
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.NotificationManager">

        <!-- The only Activity in this single-activity app -->
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:theme="@style/Theme.NotificationManager">
            <intent-filter>
                <!-- This activity is the launch point from the home screen -->
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

> ⚠ **Important — POST_NOTIFICATIONS Permission**
> On Android 13 (API 33) and higher, apps MUST declare and request POST_NOTIFICATIONS at runtime. Simply adding it to the Manifest is not enough. The app must also call requestPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS) at runtime to prompt the user. See Section 7.1 for the full implementation.

5.2 values/strings.xml — String Resources

```
<resources>
    <string name="app_name">NotificationManager</string>
    <string name="action_settings">Settings</string>
    <string name="change_language">Change Language</string>
    <!-- Navigation labels -->
    <string name="first_fragment_label">First Fragment</string>
    <string name="second_fragment_label">Second Fragment</string>
    <string name="next">Next</string>
    <string name="previous">Previous</string>
    <!-- Content strings -->
    <string name="welcome_message">Welcome to the Notification Manager app...</string>
    <string name="lorem_ipsum">Welcome to the Notification Manager app. This app
        helps you manage and test your notifications efficiently...</string>
</resources>
```

The app also provides identical string files for:

- values-hi/strings.xml — Hindi translations
- values-kn/strings.xml — Kannada translations
- values-te/strings.xml — Telugu translations

Android automatically selects the correct file based on the device or programmatically-set locale.

5.3 values/colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
</resources>
```

The app uses Material Design 3 dynamic color theming, so most colors are derived from the Material theme rather than being hardcoded. These two are available as fallback values.

5.4 values/themes.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base theme inherits from Material 3 DayNight -->
    <style name="Base.Theme.NotificationManager"
           parent="Theme.Material3.DayNight.NoActionBar">
        <!-- Custom overrides can be added here -->
    </style>

    <!-- Applied theme that extends Base -->
    <style name="Theme.NotificationManager"
           parent="Base.Theme.NotificationManager" />
</resources>
```

> 📌 **Key Concept — NoActionBar**
> 'NoActionBar' in the theme name means the default ActionBar is disabled.
> The app instead uses a MaterialToolbar widget inside an AppBarLayout,
> which is set as the ActionBar programmatically via setSupportActionBar().

5.5 values/dimens.xml

```
<resources>
    <dimen name="fab_margin">16dp</dimen>
</resources>
```

This stores the FAB's right margin as a dimension resource. Referenced in activity_main.xml as @dimen/fab_margin. Externalizing dimensions makes it easy to change spacing across the app from one file.

6. Layout Files (XML — res/layout/)

6.1 activity_main.xml — Root Activity Layout

This is the only Activity layout. It hosts the persistent UI elements: the toolbar, navigation container, and FAB.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true">

    <!-- AppBarLayout wraps the toolbar for scrolling behavior support -->
    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize" />

    </com.google.android.material.appbar.AppBarLayout>

    <!-- Includes content_main.xml which holds the NavHostFragment -->
    <include layout="@layout/content_main" />

    <!-- Floating Action Button — triggers notification -->
    <com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/fab"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom|end"
        android:layout_marginEnd="@dimen/fab_margin"
        android:layout_marginBottom="16dp"
        app:srcCompat="@android:drawable/ic_dialog_email" />

</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Component	Role
CoordinatorLayout	Root container that coordinates child views (e.g., FAB can auto-anchor to Snackbar).
AppBarLayout	Wraps the toolbar, enabling scroll-linked animations (collapse, pin, etc.).
MaterialToolbar	The actual top bar. Set as the ActionBar in code to display the title, back button, and menu.
<include>	Inserts the content_main.xml layout, keeping layouts modular.
FloatingActionButton	Anchored to bottom-right. Its click listener in Java triggers the notification flow.

6.2 content_main.xml — Navigation Container

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layout_behavior="@string/appbar_scrolling_view_behavior">

    <!-- NavHostFragment fills the entire content area -->
    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/nav_host_fragment_content_main"
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:defaultNavHost="true"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:navGraph="@navigation/nav_graph" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

> 📌 **Key Concept — NavHostFragment**

> FragmentContainerView with android:name="NavHostFragment" acts as the navigation host.
> app:navGraph="@navigation/nav_graph" links it to the navigation graph.
> app:defaultNavHost="true" means this host intercepts the system Back button.
> 0dp width/height with constraints = fill all available space.

6.3 fragment_first.xml — First Fragment Layout

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="16dp">

        <!-- Button navigates to SecondFragment -->
        <Button
            android:id="@+id/button_first"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/next"
            app:layout_constraintTop_toTopOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintBottom_toTopOf="@id/textview_first" />

        <!-- Text view shows welcome message -->
        <TextView
            android:id="@+id/textview_first"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/lorem_ipsum"
            android:layout_marginTop="16dp"
            app:layout_constraintTop_toBottomOf="@id/button_first"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintEnd_toEndOf="parent" />

    </androidx.constraintlayout.widget.ConstraintLayout>
</androidx.core.widget.NestedScrollView>
```

6.4 fragment_second.xml — Second Fragment Layout

Mirrors fragment_first.xml exactly, except the Button's id is button_second and its text is @string/previous. This demonstrates symmetric navigation between two screens.

6.5 menu/menu_main.xml — Options Menu

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">

    <!-- Standard Settings item (placeholder) -->
    <item
        android:id="@+id/action_settings"
        android:orderInCategory="100"
        android:title="@string/action_settings"
        app:showAsAction="never" />   <!-- always in overflow menu -->

    <!-- Language selector item -->
    <item
        android:id="@+id/action_language"
        android:orderInCategory="101"
        android:title="@string/change_language"
        app:showAsAction="never" />
</menu>
```

6.6 navigation/nav_graph.xml — Navigation Graph

```
<?xml version="1.0" encoding="utf-8"?>
<navigation
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/nav_graph"
    app:startDestination="@id/FirstFragment">  <!-- launches here -->

    <fragment
        android:id="@+id/FirstFragment"
        android:name="com.example.notificationmanager.FirstFragment"
        android:label="@string/first_fragment_label"
        tools:layout="@layout/fragment_first">

        <!-- Action: navigate to SecondFragment -->
        <action
            android:id="@+id/action_FirstFragment_to_SecondFragment"
            app:destination="@id/SecondFragment" />
    </fragment>

    <fragment
        android:id="@+id/SecondFragment"
        android:name="com.example.notificationmanager.SecondFragment"
        android:label="@string/second_fragment_label"
        tools:layout="@layout/fragment_second">

        <!-- Action: navigate back to FirstFragment -->
        <action
            android:id="@+id/action_SecondFragment_to_FirstFragment"
            app:destination="@id/FirstFragment" />
    </fragment>
</navigation>
```

7. Java Source Files (Top-to-Bottom Flow)

7.1 MainActivity.java — The Application Hub

This is the single Activity in the application. It is responsible for: setting up the UI, registering the notification channel, handling the FAB click (notification flow), setting up navigation, and providing the options menu.

7.1.1 Package Declaration and Imports

```
package com.example.notificationmanager;

// Android core classes
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.content.pm.PackageManager;
import android.os.Build;
import android.os.Bundle;

// Jetpack Activity / Result API
import androidx.activity.EdgeToEdge;
import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;

// Compat helpers (work on older API levels)
import androidx.core.app.ActivityCompat;
import androidx.core.app.NotificationCompat;
import androidx.core.app.NotificationManagerCompat;
import androidx.core.content.ContextCompat;

// Material Design UI
import com.google.android.material.snackbar.Snackbar;

// Navigation Component
import androidx.navigation.NavController;
import androidx.navigation.Navigation;
import androidx.navigation.fragment.NavHostFragment;
import androidx.navigation.ui.AppBarConfiguration;
import androidx.navigation.ui.NavigationUI;

// ViewBinding generated class
import com.example.notificationmanager.databinding.ActivityMainBinding;

// Localization
import java.util.Locale;
import android.content.res.Configuration;
import android.content.res.Resources;
import android.util.DisplayMetrics;
import androidx.appcompat.app.AlertDialog;
```

7.1.2 Class Declaration and Fields

```
public class MainActivity extends AppCompatActivity {

    // Unique string ID for the notification channel
    private static final String CHANNEL_ID = "main_channel";

    // AppBarConfiguration links the NavController to the ActionBar
    private AppBarConfiguration appBarConfiguration;

    // ViewBinding object — generated by Gradle from activity_main.xml
    private ActivityMainBinding binding;

    // Modern permission-request API: avoids deprecated requestPermissions()
    private final ActivityResultLauncher<String> requestPermissionLauncher =
            registerForActivityResult(
                new ActivityResultContracts.RequestPermission(),
                isGranted -> {
                    if (isGranted) {
                        sendNotification();  // user allowed → send immediately
                    } else {
                        Snackbar.make(binding.getRoot(),
                            "Notification permission denied",
                            Snackbar.LENGTH_LONG).show();
                    }
                }
            );
```

> 📌 **Key Concept — ActivityResultLauncher**
> The new Activity Result API (ActivityResultContracts.RequestPermission) replaces the

> deprecated requestPermissions(String[], int) + onRequestPermissionsResult() pattern.
> It must be registered BEFORE the Activity is created — hence it's a field, not in onCreate.

7.1.3 onCreate() — Activity Entry Point

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    // 1. Enable edge-to-edge drawing (content goes under status/nav bars)
    EdgeToEdge.enable(this);

    // 2. Inflate layout and create the binding object
    binding = ActivityMainBinding.inflate(getLayoutInflater());
    setContentView(binding.getRoot());

    // 3. Create the notification channel (safe to call every launch)
    createNotificationChannel();

    // 4. Handle window insets so content isn't hidden under system bars
    ViewCompat.setOnApplyWindowInsetsListener(binding.main, (v, insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
        return insets;
    });

    // 5. Set the toolbar as the ActionBar
    setSupportActionBar(binding.toolbar);

    // 6. Set up Navigation — find NavHostFragment and get NavController
    NavHostFragment navHostFragment = (NavHostFragment)
        getSupportFragmentManager()
        .findFragmentById(R.id.nav_host_fragment_content_main);
    NavController navController = navHostFragment.getNavController();

    // 7. Configure AppBar to work with navigation (shows Up button)
    appBarConfiguration = new AppBarConfiguration.Builder(
        navController.getGraph()).build();
    NavigationUI.setupActionBarWithNavController(
        this, navController, appBarConfiguration);

    // 8. FAB click → check permission, then send notification
    binding.fab.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
                // Android 13+ requires runtime permission for notifications
                if (ContextCompat.checkSelfPermission(MainActivity.this,
                        android.Manifest.permission.POST_NOTIFICATIONS)
                        == PackageManager.PERMISSION_GRANTED) {
                    sendNotification();
                } else {
                    requestPermissionLauncher.launch(
                        android.Manifest.permission.POST_NOTIFICATIONS);
                }
            } else {
                // Below API 33 — no runtime permission needed
                sendNotification();
            }
        }
    });
}
```

7.1.4 Options Menu Methods

```
// Called by Android to create the options menu from XML
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.menu_main, menu);
    return true;
}

// Called when the user selects a menu item
@Override
public boolean onOptionsItemSelected(MenuItem item) {
    int id = item.getItemId();
    if (id == R.id.action_settings) {
        return true;  // settings: no-op for now
    }
    if (id == R.id.action_language) {
        showLanguageDialog();  // open language picker
```

```
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
```

7.1.5 Language Switching Methods

```
    private void showLanguageDialog() {
        final String[] languages = {"English", "Hindi", "Kannada", "Telugu"};
        final String[] languageCodes = {"en", "hi", "kn", "te"};

        AlertDialog.Builder builder = new AlertDialog.Builder(this);
        builder.setTitle(R.string.change_language);
        builder.setItems(languages, (dialog, which) -> {
            setLocale(languageCodes[which]);
        });
        builder.create().show();
    }

    private void setLocale(String lang) {
        Locale myLocale = new Locale(lang);
        Resources res = getResources();
        DisplayMetrics dm = res.getDisplayMetrics();
        Configuration conf = res.getConfiguration();
        conf.setLocale(myLocale);
        res.updateConfiguration(conf, dm);  // apply configuration
        recreate();  // restart Activity to reload all resources
    }
```

7.1.6 Navigation Up Support

```
    @Override
    public boolean onSupportNavigateUp() {
        NavHostFragment navHostFragment = (NavHostFragment)
            getSupportFragmentManager()
            .findFragmentById(R.id.nav_host_fragment_content_main);
        NavController navController = navHostFragment.getNavController();
        return NavigationUI.navigateUp(navController, appBarConfiguration)
            || super.onSupportNavigateUp();
    }
```

This method is called when the user taps the Back/Up arrow in the toolbar. NavigationUI.navigateUp() pops the navigation back stack; if there's nowhere to go, it falls back to the default behavior.

7.1.7 createNotificationChannel()

```
    private void createNotificationChannel() {
        // CharSequence for the user-visible channel name
        CharSequence name = "Main Channel";
        String description = "Channel for general notifications";

        // IMPORTANCE_DEFAULT = plays sound, shown in notification tray
        int importance = NotificationManager.IMPORTANCE_DEFAULT;

        // Create the channel with CHANNEL_ID, name, and importance level
        NotificationChannel channel =
            new NotificationChannel(CHANNEL_ID, name, importance);
        channel.setDescription(description);

        // Register the channel with the system
        NotificationManager notificationManager =
            getSystemService(NotificationManager.class);
        notificationManager.createNotificationChannel(channel);
    }
```

Importance Level	Behavior
IMPORTANCE_HIGH	Heads-up notification, shows as a pop-over at top of screen.
IMPORTANCE_DEFAULT	Makes sound; appears in notification tray. Used in this app.
IMPORTANCE_LOW	No sound; appears in tray. Good for non-urgent updates.
IMPORTANCE_MIN	No sound, no visual interruption. Only visible in expanded shade.

7.1.8 sendNotification()

```
    private void sendNotification() {
        // Build the notification using the compat builder
        NotificationCompat.Builder builder =
            new NotificationCompat.Builder(this, CHANNEL_ID)
                .setSmallIcon(R.drawable.ic_launcher_foreground)
                .setContentTitle("My Notification")
                .setContentText("Hello World!")
                .setPriority(NotificationCompat.PRIORITY_DEFAULT);

        try {
            NotificationManagerCompat notificationManager =
                NotificationManagerCompat.from(this);
            // Notification ID 1 – use different IDs for different notifications
            notificationManager.notify(1, builder.build());
        } catch (SecurityException e) {
            // Catches permission-not-granted on some devices
            Snackbar.make(binding.getRoot(),
                "Notification permission required",
                Snackbar.LENGTH_LONG).show();
        }
    }
}  // end of MainActivity
```

7.2 FirstFragment.java

```
package com.example.notificationmanager;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import androidx.annotation.NonNull;
import androidx.fragment.app.Fragment;
import androidx.navigation.fragment.NavHostFragment;
import com.example.notificationmanager.databinding.FragmentFirstBinding;

public class FirstFragment extends Fragment {

    private FragmentFirstBinding binding;   // ViewBinding for this fragment

    // Step 1: Inflate the fragment layout
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
                             ViewGroup container,
                             Bundle savedInstanceState) {
        binding = FragmentFirstBinding.inflate(inflater, container, false);
        return binding.getRoot();
    }

    // Step 2: Set up click listeners after views are created
    @Override
    public void onViewCreated(@NonNull View view, Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        // Lambda: navigate to SecondFragment using the action ID from nav_graph.xml
        binding.buttonFirst.setOnClickListener(v ->
            NavHostFragment.findNavController(FirstFragment.this)
                .navigate(R.id.action_FirstFragment_to_SecondFragment)
        );
    }

    // Step 3: Clear binding reference to prevent memory leaks
    @Override
    public void onDestroyView() {
        super.onDestroyView();
        binding = null;
    }
}
```

> ⚠️ **Important — Why binding = null in onDestroyView()?**
> Fragments outlive their views. The binding holds a reference to the view hierarchy.
> If binding is NOT cleared in onDestroyView(), the Fragment will hold the entire
> view tree in memory even when the fragment is not displayed, causing a memory leak.
> Always set binding = null in onDestroyView() when using ViewBinding in fragments.

7.3 SecondFragment.java

```
public class SecondFragment extends Fragment {

    private FragmentSecondBinding binding;

    @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
                             ViewGroup container,
                             Bundle savedInstanceState) {
        binding = FragmentSecondBinding.inflate(inflater, container, false);
        return binding.getRoot();
    }

    @Override
    public void onViewCreated(@NonNull View view, Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        // Navigate BACK to FirstFragment
        binding.buttonSecond.setOnClickListener(v ->
            NavHostFragment.findNavController(SecondFragment.this)
                .navigate(R.id.action_SecondFragment_to_FirstFragment)
        );
    }

    @Override
    public void onDestroyView() {
        super.onDestroyView();
        binding = null;
    }
}
```

8. Complete Lifecycle Flow (Summary)

8.1 Full App Execution Order

#	Event / Method	File	What Happens
1	App launch (MAIN intent)	AndroidManifest.xml	OS finds .MainActivity as LAUNCHER.
2	onCreate()	MainActivity.java	ViewBinding, channel creation, nav setup, FAB listener.
3	NavHostFragment loads	content_main.xml, nav_graph.xml	NavController inflates nav graph; FirstFragment is displayed.
4	FirstFragment.onCreateView()	FirstFragment.java	fragment_first.xml inflated via FragmentFirstBinding.
5	FirstFragment.onViewCreated()	FirstFragment.java	button_first click listener registered.
6a	User taps Next button	FirstFragment.java	navigate(action_First_to_Second) called.
6b	User taps FAB	MainActivity.java	Permission check; sendNotification() or request permission.
6c	User taps Language menu	MainActivity.java	AlertDialog shown; setLocale() then recreate().
7	SecondFragment lifecycle	SecondFragment.java	Same as steps 4-5 but for SecondFragment.
8	Back / Previous button	SecondFragment.java	NavController pops back stack → FirstFragment.
9	onDestroyView() (fragments)	First/SecondFragment.java	binding = null to prevent memory leak.

8.2 Notification Flow (Detailed)

```
User taps FAB
     |
     ├─ Build.VERSION.SDK_INT >= 33 (TIRAMISU)?
     |       |
     |   YES |  ContextCompat.checkSelfPermission(POST_NOTIFICATIONS)
     |       |       |
     |       ├── GRANTED?  → sendNotification()
     |       └── DENIED?   → requestPermissionLauncher.launch(POST_NOTIFICATIONS)
     |                           |
     |                       System dialog shown to user
     |                           |
     |                       isGranted callback fires
     |                           ├── true  → sendNotification()
     |                           └── false → Snackbar("permission denied")
     |
     └─ SDK < 33? → sendNotification() directly

sendNotification():
     NotificationCompat.Builder → setSmallIcon, setTitle, setText, setPriority
     NotificationManagerCompat.from(this).notify(1, builder.build())
     ──────────────────────────────────────────────────────────────
     System Notification Tray: shows "My Notification" / "Hello World!"
```

9. File Relationship Diagram

The following diagram shows how files depend on each other at runtime:

```
AndroidManifest.xml
  └── declares MainActivity → triggers app launch
        │
        ├── android:theme → values/themes.xml
        │       └── parent: Theme.Material3.DayNight.NoActionBar
        │
        └── POST_NOTIFICATIONS permission used by MainActivity.java

MainActivity.java
  ├── uses ActivityMainBinding (generated from activity_main.xml)
  │       └── activity_main.xml
  │               ├── CoordinatorLayout > AppBarLayout > MaterialToolbar (id: toolbar)
  │               ├── <include> content_main.xml
  │               │       └── FragmentContainerView (id: nav_host_fragment_content_main)
  │               │               └── app:navGraph → nav_graph.xml
  │               │                       ├── FirstFragment  → FirstFragment.java
  │               │                       └── SecondFragment → SecondFragment.java
  │               └── FloatingActionButton (id: fab)
  │
  ├── menu inflated from menu/menu_main.xml
  │       ├── action_settings (no-op)
  │       └── action_language → showLanguageDialog() → setLocale()
  │
  ├── CHANNEL_ID "main_channel" → createNotificationChannel()
  └── sendNotification() → uses R.drawable.ic_launcher_foreground

FirstFragment.java
  ├── uses FragmentFirstBinding (generated from fragment_first.xml)
  │       ├── Button id: button_first (text: @string/next)
  │       └── TextView id: textview_first (text: @string/lorem_ipsum)
  └── navigate → R.id.action_FirstFragment_to_SecondFragment (nav_graph.xml)

SecondFragment.java
  ├── uses FragmentSecondBinding (generated from fragment_second.xml)
  │       ├── Button id: button_second (text: @string/previous)
  │       └── TextView id: textview_second
  └── navigate → R.id.action_SecondFragment_to_FirstFragment (nav_graph.xml)

String Resources used at runtime:
  values/strings.xml          (default — English)
  values-hi/strings.xml       (Hindi locale)
  values-kn/strings.xml       (Kannada locale)
  values-te/strings.xml       (Telugu locale)
```

10. Lab Exercises for Students

Exercise 1 — Run & Observe (Beginner, ~20 min)

Objective: Verify the development environment and understand the basic application.

8. Clone/open the NotificationManager project in Android Studio.
9. Build and run on an emulator or physical device (API 26+).
10. Tap the FAB (mail icon). Observe the notification in the system tray.
11. Tap "Next" and "Previous" to navigate between fragments.
12. Open the three-dot menu and switch to Hindi. Observe the UI change.

> ✏️ **Record Your Observations**
> 1. What icon appears in the notification? Why is it the launcher icon?
> 2. What happens to the toolbar title when you navigate to SecondFragment?
> 3. After switching to Hindi, does the notification text also change? Why or why not?

Exercise 2 — Modify Notification Content (Beginner–Intermediate, ~30 min)

Objective: Understand how to customize notification appearance.

13. In MainActivity.java, change "My Notification" to your own title.
14. Change "Hello World!" to a longer message.
15. Add .setStyle(new NotificationCompat.BigTextStyle().bigText(...)) for expandable text.
16. Change the importance from IMPORTANCE_DEFAULT to IMPORTANCE_HIGH in createNotificationChannel().
17. Run the app and describe what is different about the notification behavior.

Exercise 3 — Add a Third Fragment (Intermediate, ~45 min)

Objective: Practice extending the navigation graph.

18. Create a new layout file res/layout/fragment_third.xml with a Button and TextView.
19. Create ThirdFragment.java extending Fragment, using ViewBinding.
20. Add the ThirdFragment to nav_graph.xml with a navigation action from SecondFragment.
21. In SecondFragment.java, navigate to ThirdFragment when the existing button is clicked.
22. In ThirdFragment.java, navigate back to FirstFragment on button click.
23. Verify the complete circular navigation: First → Second → Third → First.

Exercise 4 — Notification with PendingIntent (Advanced, ~60 min)

Objective: Learn how to navigate from a notification to a specific screen.

24. In MainActivity.java, create a PendingIntent that opens the app when the notification is tapped.
25. Use Intent + PendingIntent.getActivity() with FLAG_IMMUTABLE.
26. Add .setContentIntent(pendingIntent) and .setAutoCancel(true) to the notification builder.
27. Test: tap the notification when the app is in the background. Verify the app opens.

Exercise 5 — Persistent Locale Setting (Advanced, ~60 min)

Objective: Use SharedPreferences to remember the selected language.

28. In setLocale(), save the language code using SharedPreferences.
29. In onCreate(), read the saved language code and apply it before inflating views.
30. Test: change to Hindi, force-close the app, reopen. The language should persist.

Exercise 6 — Dark Mode Support (Advanced, ~45 min)

Objective: Understand night-mode theming.

31. Examine values-night/themes.xml.
32. Add a custom color override (e.g., colorPrimary) for dark mode.
33. Add a "Toggle Theme" option to the menu that calls AppCompatDelegate.setDefaultNightMode().
34. Test in both light and dark mode.

11. Prerequisites Checklist

11.1 Software Requirements

Software	Minimum Version	Where to Download
Android Studio	Meerkat (2024.3.2+)	developer.android.com/studio
JDK (Java Dev Kit)	JDK 11 or 17	Bundled with Android Studio
Android SDK	API Level 26–36	Via SDK Manager in Android Studio
Android Emulator OR	API 26+ system image	Via AVD Manager in Android Studio
Physical Device	Android 8.0+	Enable USB Debugging in Developer Options
Gradle (wrapper)	9.4.1 (auto-downloaded)	Managed by gradle-wrapper.properties

11.2 Knowledge Prerequisites

Students should be familiar with:

- Java programming — classes, methods, interfaces, anonymous classes, lambdas.
- Object-Oriented Programming — inheritance, polymorphism, encapsulation.
- XML syntax — tags, attributes, namespaces.
- Basic Android concepts — Activity, Fragment, Intent, Context, lifecycle methods.
- Android Studio IDE — project creation, running on emulator, Logcat.

11.3 Environment Setup Steps

35. Install Android Studio from the official website.
36. On first launch, complete the setup wizard (installs SDK, emulator, system image).
37. Unzip the NotificationManager project folder.
38. In Android Studio: File → Open → select the NotificationManager folder.
39. Wait for Gradle sync to complete (downloads dependencies automatically).
40. Create or start an AVD (API 26+) using Tools → Device Manager.
41. Click Run ▶️ to build and deploy the app.

12. Troubleshooting

Problem	Likely Cause	Solution
Gradle sync fails	No internet / proxy issue	Check internet; disable VPN; File → Invalidate Caches → Restart.
App crashes on launch (NullPointerException)	NavHostFragment not found or binding null	Ensure nav_host_fragment_content_main ID matches content_main.xml.
Notification does not appear	Permission denied or channel not created	API 33+: Grant POST_NOTIFICATIONS in Settings → Apps. Also verify CHANNEL_ID matches in createChannel() and sendNotification().
FAB click does nothing on API 33+	Permission not yet granted	First tap triggers permission dialog. Grant it. Subsequent taps send notifications.
Language does not change	Missing translation file or wrong locale code	Verify values-hi/, values-kn/, values-te/ all exist with correct strings.xml.
"Cannot find symbol: ActivityMainBinding"	ViewBinding not enabled or build not run	Ensure buildFeatures { viewBinding = true } in app/build.gradle.kts. Run Build → Make Project.
Navigation back arrow missing	AppBarConfiguration or NavController not set up	Confirm NavigationUI.setupActionBarWithNavController() is called in onCreate().
Emulator very slow	Hardware acceleration disabled	Enable HAXM (Intel) or WHPX (Windows), or use API 30+ with ARM system image on Apple Silicon.
"minSdk" version error	Emulator API below 26	Create a new AVD with API 26 or higher in the AVD Manager.

12.1 Useful Logcat Filters

In Android Studio's Logcat window, use these filters to debug the app:

Filter	What It Shows
tag:MainActivity	Log messages from MainActivity (add your own Log.d() calls).
tag:NotificationManager	System notification-related messages.
level:Error	Crashes and errors only — fastest way to find exceptions.
package:com.example.notificationmanager	Shows only messages from this application.

—— End of Lab Manual ——

Department of Artificial Intelligence And Data Science | Mobile Application Development